riverrun

The Anonymity Layer for Solana, Post-Quantum

Hide *who* acted, not just how much you moved.
One secret, a different identity in every context, and nothing a quantum computer can break.

riverrun, past Eve and Adam's, from swerve of shore to bend of bay,
brings us by a commodius vicus of recirculation.
— James Joyce, *Finnegans Wake*, first line

Abstract. *On a public blockchain, hiding* who *did something is a different problem from hiding* how much *money moved — and a harder one to reason about honestly. This paper describes* RIVERRUN*, a set of tools for the first problem on Solana. It has three parts. A* ***ruler*** *measures how much anonymity a privacy pool really gives its users, by tracing where their money came from; run against a live pool on Solana mainnet, a set that advertises* 30 *members delivers an effective* 6.5*, and one member is alone. An* ***identity layer****,* riverrun ID*, turns one secret into a different, unlinkable identity in every app, while still stopping one person from acting twice where they should act once. And a* ***pool*** *lets a member commit an intent and later carry it out, with no observer able to say which member did it, proven by a transparent, post-quantum STARK with no trusted setup. Everything is built from one hash, so a quantum computer does not break it. Every "X is bound" claim has a test that we deleted the constraint to watch fail; three such tests turned out to prove nothing, and we say which. Where the system is not finished — an unaudited circuit, on-chain verification proven in a local VM but not yet on mainnet — we say so plainly. Every number here comes from one command.*

Manuel Guilherme Galmanus • Independent Researcher • Preprint, July 2026

Privacy is not secrecy. Secrecy is hiding what you did from everyone; privacy is the power to choose who sees it. On a public ledger you have neither by default: every action you take is pinned to a board that never forgets, and firms you never hired read it for a living. No exchange, no analytics company, no protocol will hand you behavioral privacy out of kindness. If we want it, we build it — and we build it to outlast the machine coming to break it, hash-only, no trusted setup, no one to ask permission from. The cypherpunks wrote code to defend privacy. We write code, and then we do the one thing the old manifestos left to faith: we do not ask you to trust that the privacy is real. We hand you a ruler, and you measure it yourself.

Contents

1 The problem, in plain words

Picture a town where every letter anyone ever mailed is pinned to a board in the square, forever. Not the contents — those are sealed — but the envelopes: who sent what to whom, when, and how heavy it was. You would learn a great deal about that town without opening a single letter. Who pays whom. Who is suddenly buying. Who moves money at three in the morning. Who copies whose move an hour later.

A public blockchain is that board. People think that a wallet address, being a random string and not a name, keeps them private. The opposite is closer to the truth. The address is a fixed handle, and everything done under it piles up into a pattern. How much you move, which apps you touch, in what order: modern chain-analysis uses that pattern to link your wallets, attach a name, and increasingly to predict and front-run what you will do next. These tools get better every month, because the data is permanent and public and the models feeding on it are hungry.

Most privacy work assumes the problem is the money: hide the amounts, hide the balances. That view is incomplete. There are two kinds of privacy here, and they are not the same. One hides *how much*. The other hides *who* — which person, out of a crowd, did a public thing. Almost all the work so far is on the first: confidential transfers, shielded balances, mixers that break the link between a deposit and a withdrawal of value. This paper is about the second. Amounts are not hidden here and are not the point. What we cut is the link between an *actor* and an *action*.

Here is the whole idea in one sentence, and the rest of the paper is only this sentence made precise and made real: **you disappear into a crowd of people who all look like they could have done the thing you did.**

On the name. *Finnegans Wake* (James Joyce, 1939) is a novel written as a circle: its last sentence breaks off mid-phrase and runs straight into its first word, "riverrun." We did not pick the name for the poetry. One idea from the book is load-bearing and shows up later as a number: a circle has no beginning, and a funding trail that loops back on itself has no origin to trace (§8). That is the strongest defense the ruler finds.

2 What riverrun is

RIVERRUN is not one program. It is three things that fit together, and it helps to name them before the details.

1. **A ruler** (§8). It measures how much anonymity a privacy pool *really* gives, instead of the number the pool advertises. It reads only public chain data, works on pools it has never seen, and **runs on Solana mainnet today**. This is the finished, useful-now part.
2. **An identity layer**, *riverrun ID* (§3). One secret becomes a different, unlinkable identity in every app, while still letting each app enforce "one action per person." This is Solana's missing Semaphore, and it is post-quantum. The primitive is built and tested; the on-chain layer is the road ahead.

3. **A pool** (§4–§7). A member commits an intent and later carries it out unlinkably, proven by a post-quantum STARK. It is one application of the identity layer, and the research core of the system.

The through-line is one design choice made everywhere: **build from a single hash function, and nothing else.** No elliptic curves, no pairings, no trusted setup ceremony. That is what makes the whole thing *post-quantum* — a large quantum computer breaks the curve-based cryptography under most of today's privacy tools, but it does not break a hash. Section 2.1 says exactly why that matters now and not in ten years.

2.1 Why a tool like this must be post-quantum — not *may*, *must*

The usual objection is that a quantum computer big enough to break today's cryptography does not exist yet. True, and beside the point. The argument for post-quantum here is not about fear; it is an inequality, due to Mosca. Let X be how long a secret must stay secret, Y the time to migrate a system to quantum-safe cryptography, and Z the time until a cryptographically-relevant quantum computer exists. If

$$X + Y > Z,$$

the machine arrives before the protection does, and everything recorded in the meantime is decrypted retroactively.

Now put a blockchain anonymity set into that inequality. The ledger is public and permanent: the link between an actor and their action, if it survives at all, survives forever, so $X = \infty$. No finite Y or Z satisfies the inequality. For a privacy tool whose data lives on a permanent public ledger, $X + Y > Z$ is not a risk to manage; it is a *certainty* — unless the primitive is *already* quantum-safe at the moment of writing. This is what **harvest-now-decrypt-later** means made precise: an adversary needs no quantum computer today, only a copy of the chain, which is free; the day the hardware exists, every curve-based guarantee ever written to that chain fails at once, including the ones written in 2026.

Disanalogy, stated. For ephemeral secrets — a TLS session key, a monthly-rotated password — X is small, and a non-post-quantum scheme is defensible for a few more years. That is exactly the reasoning that does not transfer here: on-chain anonymity has no expiry, so the comfort ephemeral data enjoys does not apply.

Curve-based privacy tools fail the test concretely: Shor's algorithm breaks the curve, and the recorded data is already in the adversary's hands. Hash-based tools pass it: the best known quantum attack on a hash is Grover's, which only halves the security level, and a 256-bit hash absorbs that with room to spare. RIVERRUN is hash-only by design, so a user who hides a behavior today is still hidden after quantum hardware exists — at no extra cost, because the property is structural, not bolted on. The standards bodies have already acted on this for data far less permanent than a ledger: NIST finalized its post-quantum standards (FIPS 203/204/205) in August 2024, and its hash-based signature, FIPS 205 (SLH-DSA), rests on the same conservative assumption RIVERRUN does. Read the other way: in 2026, a permanent-ledger privacy tool that is *not* post-quantum is shipping a guarantee it already knows will expire.

3 riverrun ID: one secret, seven powers

Start with the picture that started it. Take a jigsaw piece. It has a shape. Turn it over and the back has a different shape. Turn it to a new angle and it fits a different neighbor. *One piece, many shapes, each shape its own matching fit.* riverrun ID is that piece, in cryptography.

A user holds one **secret** s. A **context** — call it an angle θ — is any public label: an app, a DAO, an airdrop, a voting round. From the one secret and a context, the user derives an identity for *that* context and nothing else. Same secret, different angle, a different and unlinkable identity, each with its own matching fit. Seven powers come out of this, and every one is a hash plus, in one case, a little polynomial arithmetic — so all seven are post-quantum.

#	power	what the user can do
1	**shape**	be a different, unlinkable identity in every context
2	**fit**	act *once* per context — sybil-resistant
3	**turn**	prove "I am the same person across two rounds" in zero knowledge, revealing to no one *which* person
4	**link**	choose to reveal that two of your identities are one — to whom you pick, for the contexts you pick
5	**grant**	lend one context to an agent, bound to that agent, scoped to that context, never handing over your master secret
6	**rln**	rate-limit yourself: N actions stay anonymous, the N+1-th lets anyone unmask you
7	**credential**	carry an issuer's attribute ("verified", "over 18") and show it per context, without a trail

Put together, the user is **invisible by default** (1), **accountable where it matters** (2, 6), **continuous when they choose** (3), **linkable only on their own terms** (4), able to **lend a single context to an agent** (5), and able to **prove a fact about themselves with no trail** (7). One secret, full control of your own exposure, quantum-safe throughout.

> Two powers carry the weight. **shape** makes you unlinkable: your identity in a DAO and your identity in an airdrop are two different hashes of the same secret, and no one can tell they belong to one person. **fit** makes you accountable: it is a token you can spend only once in a context, so "one vote per member" or "one claim per person" holds *without* anyone learning who you are. Invisible and one-per-person at the same time — that pair is the whole trick, and everything else is a refinement of it.

3.1 What it unlocks

Each of these is just a different context — a different angle on the same secret:

- **Anonymous DAO voting** — one vote per member, nobody learns who voted how.

- **Sybil-resistant airdrops** — one claim per person, unlinkable to their main wallet.
- **Portable, unlinkable reputation** — prove "verified member" across apps with no trail joining them.
- **Rate-limited anonymous posting** — act under a per-context cap, spam priced in your own de-anonymization.
- **Delegation to agents** — let a bot act as you in *one* app, without giving it the keys to the rest of your life.

3.2 Honest positioning

This is not a new idea in concept. On Ethereum it is essentially **Semaphore**: a secret identity plus scoped nullifiers gives unlinkable per-context identities with one-action-per-person, and Worldcoin and RLN build on it. riverrun ID does not claim to invent that. What is genuinely uncommon is the *combination*: a **post-quantum** (hash-only, no curves) identity layer, on **Solana**, which has no Semaphore-equivalent today, paired with a **ruler** that tells you how real each context's anonymity actually is. The honest one-liner:

> *Solana has no Semaphore. riverrun ID is a post-quantum, transparent one, with a ruler that reports your real anonymity per context.*

The seven-power primitive is built and tested in the core crate (44 tests), and the "turn" power is proven in zero knowledge over the same STARK the pool uses. What is *not* yet built is the in-circuit membership proof over a small field, its Solana verifier, and an audit. Those are integration work on vetted parts (Plonky3's Poseidon2, the M31 verifier class of §11), not new mathematics — but they are not done, and this section is the product's north star, not a shipped claim.

4 The pool: commit an intent, act unlinkably

Tornado Cash, the Ethereum mixer, breaks the link between putting money *in* and taking money *out*. You deposit one coin; later, from a fresh address, you withdraw one coin, proving you are owed a withdrawal without revealing which deposit was yours. The pool is a crowd, and you hide in it.

RIVERRUN keeps the shape and changes what is hidden. Instead of a coin, the hidden thing is a *behavior*. Three steps.

Commit. A member publishes $\ell = \text{Rescue}(s, a)$ into a public set, where s is a secret only they know and a is the action they intend — "swap 1 SOL for USDC on venue V", encoded as a hash. Publishing ℓ announces that *somebody in this set intends action* a, and nothing about who. The set is a Merkle tree; its root R is the public fingerprint of the whole crowd.

Execute. Later — ideally in a round where many members act at once — the member does a from a fresh identity, attaching a zero-knowledge proof of one sentence: *"I know a secret* s *whose commitment* $\text{Rescue}(s, a)$ *sits under the public root* R*, and my nullifier this round is* $n = \text{Rescue}(s, r)$*."* The action is public. The author is not.

Settle. The pool checks the proof, checks the nullifier n is fresh this round, spends it, and releases the action. The public record is $\{R, a, n\}$, with no link back to any commitment.

Two questions decide whether this is real privacy or theater, and both are about the nullifier. Why have one? Because without it a member could act a thousand times from a thousand fresh identities, and the "crowd" would be one person in a thousand masks. The nullifier is a token derived from the secret and the round, spendable once per round. It reveals nothing about s, and one member's nullifiers in different rounds cannot be tied together. So it enforces "one action per member per round" while never naming the member. This is a studied primitive; PLUME [6] formalizes exactly it.

The harder question: how does anyone know the revealed n came from the *same* secret that proved membership? If those two facts are proven separately, a member could prove membership honestly and then present a stranger's nullifier — spending someone else's turn, or hiding that they spent their own. The two halves have to be welded inside one proof. Getting that weld right, and *knowing* it is right, is the next section.

5 The proof: one object, three bindings

A zero-knowledge proof convinces you a statement is true while telling you nothing beyond its truth. Two families are used on blockchains, and the choice is a real fork.

Pairing-based SNARKs (Groth16) give tiny proofs — a few hundred bytes — that verify cheaply. Their price is a *trusted setup*: a one-time ceremony makes a secret proving key, and if that secret ever leaks, the whole system can be forged. The key is tens of megabytes that must be produced, trusted, and shipped to every prover. And the security rests on elliptic curves, which a quantum computer breaks.

STARKs are built from a hash function alone. No pairings, no ceremony, no per-circuit setup; security rests on the hash, believed to survive a quantum adversary. The price is proof size: kilobytes, not bytes. RIVERRUN takes this side on purpose, and §11 measures exactly what it costs.

What a STARK is, in one paragraph. Write your computation as a big table: rows are time-steps, columns are registers, arranged so "it ran correctly" becomes "every pair of neighboring rows obeys a fixed rule." Turn each column into a polynomial. The rules become equations those polynomials must satisfy. The prover commits to the polynomials; the verifier challenges them at random points. A cheat would need a *wrong* low-degree polynomial that agrees with a right one almost everywhere, which is impossible unless they are equal. The verifier never sees the answers, only a few random cells and a proof that all the cells are consistent — which is why the witness stays hidden.

5.1 What the circuit proves

RIVERRUN uses a STARK over a Rescue-Prime [12] Merkle tree — a hash chosen because it is cheap to write as polynomial constraints. The proof's public inputs are the four values $\{R, n, r, a\}$: root, nullifier, round, action. Its private inputs — the witness — are the secret s, the leaf's position, and the Merkle path. One secret must satisfy all three of these at once:

1. **Membership.** Rescue(s, a) is a leaf under R, via the private path.
2. **Nullifier binding.** n = Rescue(s, r): the revealed nullifier is this member's, this round.
3. **Action binding.** The leaf under R commits to the public action a: the member does the intent they registered, not another.

Because all three read the same private s, no member can mix and match. That is the weld.

5.2 Knowing the weld holds

A privacy proof that is subtly wrong is worse than no proof: it advertises a guarantee it does not deliver. So the interesting question is not "did we write the constraints" but "do they bite." The method used throughout RIVERRUN is *mutation testing*: for every claim "X is bound" there is a test, and we delete the constraint meant to enforce X and confirm the test then fails. If deleting it changes nothing, the test was passing for some other reason and is worthless.

This is not a formality. Three tests written during development were exactly that worthless, and mutation testing is how they were caught. The action-binding tests, for one, passed even with the binding deleted, because the action also feeds the proof's transcript, so a mismatched public input was rejected for an unrelated reason. The real attack hashes the committed action into the leaf (so the path still resolves) while *announcing* a different one; delete the constraint and that forged proof verifies. The useful test performs that attack. Two other tests — a nullifier check and, in the ricorso, a whole migration check — had the same disease and the same cure.

> None of this makes the circuit audited. It is hand-rolled, and a passing negative test is necessary, not sufficient, for soundness. An independent review is on the roadmap. What mutation testing removes is one specific, common, embarrassing failure: tests that look like they prove something and prove nothing.

5.3 A leak we found by measuring

The first version of the circuit carried the secret in two columns held *constant* across the whole trace. This looked harmless. It was not. A STARK opens the prover's data at random points, and a constant column has a constant low-degree extension — so the secret showed up, verbatim, in nearly every opening. We knew only because a test checked, over many proofs, whether the raw secret bytes appeared in the proof. They did: twenty times out of twenty. Confining the secret to the eight rows where it is actually needed fixed it: zero out of twenty. A single-sample test would have missed it. This is why we say "the witness is not on the wire" and *not* "zero-knowledge": the prover does not randomize the witness, so formal zero-knowledge is a claim we decline to make.

6 One crowd, one proof

The privacy argument rests on a crowd. If k members do the same action in the same round, an observer's chance of pinning any one execution is at best $1/k$. The natural way to run this is a synchronized round: many members acting identically at one moment, so there is no timing and no ordering to tell them apart.

A small observation with a big payoff: that synchronized round is exactly the shape that lets many proofs collapse into one. A STARK is a statement about an execution table. The k members' tables are independent, so they lay end to end into one longer table, with a *seam* at each boundary that switches off the linking logic so one member's last row does not bleed into the next member's first. One proof for the whole round, one verification.

members k	one batched proof	k separate proofs	saving
4	19,772 B	45,300 B	2.3×
64	43,684 B	1,074,169 B	24.6×

At k=64 a per-member scheme ships over a megabyte of proof and needs 64 verifications; the batched round is one proof, one verification.

Two limits. The prover of a batched round must know every member's secret. For a crowd of distrustful strangers that is a custodian, not privacy, and would need distributed proving. But for one operator running k identities — an agent fleet, a market maker — one prover already holds all k secrets, and the 24.6× is real. And batching trades prover time for proof size: the table is k times longer and proving is super-linear, so a round costs more wall-clock than the sum of separate proofs. It is the right trade when the scarce resource is on-chain verifications.

7 The ricorso: the crowd starts over

The name earns its keep here. *Finnegans Wake* runs on a cycle that returns to its start. RIVERRUN borrows that: the anonymity set is reborn each cycle.

The leak this closes is easy to miss. A member's per-round nullifier changes every round, so one execution is unlinked from another. But their *leaf*, Rescue(s, a), never changes: published once at commit, it stays forever. Two consequences. Anyone who ever learns your leaf — a bug, a subpoena, a careless client — links you across every round you ever acted in; the nullifiers protect executions from each other, but nothing protects a member from their own history. And a set that only grows is a public timeline of who joined when.

The ricorso lets the set start over each cycle. A member holds a fresh secret per cycle, s_c = Rescue$(s, c, \mathsf{dom}_{\text{cycle}})$, and a fresh leaf ℓ_c = Rescue(s_c, a), and at each rebirth proves the new leaf descends from *some* leaf under the previous root — without revealing which. A migration nullifier, spent once per cycle, stops one seat from becoming several. The two domain tags must differ, or publishing a migration nullifier would leak the next cycle's secret; a test pins that. History stops piling up.

The relation is built and mutation-tested *in the clear*; the STARK that proves it without the witness is specified, not built. The reason is concrete: the migration circuit needs five hash cycles before the path, which shifts valid set sizes to $\{8, 2048\}$ while the execution circuit needs

{4, 64, 16384}, and the two do not overlap. Shipping it means realigning both — a day on the most delicate code, and the honest place to stop, with the execution path green, is at the specification. It is exactly where the nullifier binding stopped before it was later built and verified.

8 The ruler: your k is not your k

Everything so far builds the pool. This section *measures* it, and it applies not just to RIVERRUN but to every anonymity pool of this kind. It is the most useful part of the paper, because it names a number that is quietly wrong across a whole category of systems.

8.1 The advertised number counts the wrong thing

Every pool reports its privacy as $1/k$: k members, so a one-in-k guess. That counts *members*, and assumes they are interchangeable. They are not, because of where their money came from. Every member funded their wallet somehow, and the funding trail is public. Trace it backward and you often reach an attributable origin: an exchange's withdrawal address, a known service, a doxxed funder. Sort the crowd by these origins. If the acting identity's origin is visible too — and a fresh withdrawal wallet has to be funded from *somewhere* — then learning it does not leave k suspects. It leaves only the members who share that origin. The crowd you are hidden in is your *provenance class*, not the whole set.

This is the same shape as the paper's opening point. There is a *surface everyone watches* (the advertised k) and a *larger real surface behind it* (the effective k, set by the funding graph). Guarding the first while ignoring the second is the mistake — and here we can measure exactly how big the gap is.

The measure itself is not new. Serjantov and Danezis [1] showed in 2002 that the honest quantity is not set size but the *entropy* of the distribution linking suspects to the event. Measuring advertised-versus-real anonymity is an established programme on Ethereum [2, 3, 4, 5]. Two things are new here: the channel (funding provenance, which those works do not condition on) and the chain (none of it has run on Solana).

8.2 The formula, and why its direction matters

Split the k members into provenance classes of sizes $n_1, n_2, \ldots$. The adversary learns the actor's class c with probability n_c/k, then guesses uniformly among that class's n_c members. The uncertainty left over is the average over classes of the log of the class size:

$$H = \sum_c \frac{n_c}{k} \log_2 n_c, \qquad k_{\text{eff}} = 2^H. \tag{1}$$

k_{eff} is the size of the crowd the member is genuinely hidden in. One class holding everyone gives $\log_2 k$: nothing was split. All-singleton classes give $H = 0$, an effective k of one: every member identified.

Worked by hand. Ten members: six funded from exchange A, three from B, one alone. Advertised anonymity 1/10. Condition on origin. With probability 6/10 the actor is an "A" and the adversary cannot tell which of six: $\log_2 6 \approx 2.58$ bits. With 3/10, a "B": $\log_2 3 \approx 1.58$. With 1/10, the lone member: 0 bits, identified. Average: $H = 0.6(2.58) + 0.3(1.58) + 0.1(0) = 2.02$ bits, so $k_{\text{eff}} \approx 4.1$. The pool sold ten and delivered four — and the lone member has an anonymity of exactly one, no matter how big the crowd behind them.

The *direction* is the whole thing, and the first implementation had it backwards. It is tempting to measure how *concentrated* the classes are, but that reports catastrophe precisely when the measurement found nothing to split on: a single class of seven — the happy case — came out as "effective $k = 1$", when the honest reading is "effective $k = 7$." Following Serjantov and Danezis, we use the residual entropy of Eq. 1 directly. Four tests pin the direction.

8.3 The measurement, on live mainnet

We ran this against a live Tornado-style SOL privacy pool on Solana mainnet, sampling 30 real depositors and tracing each funding graph backward with a bounded, public-RPC walk.

depositors sampled	30
reach an attributable origin	11/30 (37%), mean 1.18 hops
provenance classes	12
advertised k	**30**
effective k	**6.5**
worst case	**1**

A set advertising 30 delivers an effective 6.5. One depositor sits alone in their class: for them the pool provides no anonymity against an adversary who reads the funding graph, while its dashboard reports one-in-thirty.

Three honesties. It is not a flaw in that pool: no deposit-pool design controls where its users' money came from, which is exactly why the channel goes unmeasured and keeps working. It is a *floor*: a public RPC following only SOL to bounded depth sees less than a funded analyst with a tag database, so the real leak is at least this large. And 37% sits in the same 20–35% range that the Ethereum literature [4] finds with behavioral heuristics on Tornado — a sanity check on the tool, not the pool.

The ruler runs against mainnet today. A single-wallet trace, verbatim from a run over the public mainnet-beta endpoint:


```
{"endpoint":"...mainnet-beta...", "attributable_origin":false, "trace_depth":3, "reliable":true,
"rpc_calls":1, "rpc_failures":0}
```


A deeper 30-depositor audit on the same free endpoint comes back `reliable:false` with a nonzero `rpc_failures`, because the public RPC rate-limits it. The tool marks such a result

partial rather than reporting a smaller number as fact — a rate-limited or unreachable RPC can never be mistaken for a clean "private" answer. A paid endpoint completes the deep audit.

8.4 The same ruler, turned on RIVERRUN — and on the user

Measuring others' pools and not your own is marketing. The same function runs over RIVERRUN's own constructions, against 2000 synthetic members. There are three, forming a ladder of increasing privacy:

construction	reaches an origin	"which origin"	k_{eff} of 2000
rooted decoy (the field)	100%	0 bits	500
cyclic, **ambiguous** origin	100%	**2.90** bits	2000
cyclic, **rootless**	0%	—	2000

The first row is what the field ships: every trail ends at one attributable origin, so there is no doubt where the money came from, and the crowd collapses to 500. The third row is the circularity idea — a funding cycle with no beginning, so there is no origin to trace, and the set is worth its full 2000.

The middle row is the strongest one in practice. The funding trail *does* reach origins — 100% of the time — but it reaches *many*, arranged so none is privileged, so "which is the real origin" is not hidden, it is *undefined*. We measure 2.90 bits of doubt about which, and the set stays worth its full 2000. This matters because rootlessness is an idealization: it holds only while a construction's funding sources are themselves unattributable, and the moment one is a known exchange, the "no origin" claim fails. But it does not fail to nothing — it degrades exactly to *ambiguity*. Ambiguity is where rootlessness lands when it meets the real world, and it is where the defense actually lives.

Ambiguity defends a specific thing: attribution to a *particular* origin, not membership in the *group* of origins. If an adversary needs only "this came from one of these three exchanges" rather than which, ambiguity does not stop them. It buys 2.90 bits of "which", not membership secrecy.

The most useful turn is toward the user. The measurement above audits a pool after the fact. Flip it: before a user deposits into *any* pool, trace *their* wallet, sample the pool's depositors, and tell them the anonymity *they* personally would get. The tool is protocol-agnostic — it reads public chain data, not the pool's internals — so it works against a program it has never seen. Three verdicts, each with an action: **Exposed** (alone in your class; fund from a common origin or wait), **Weak** (a small minority shares your class), **OK** (a healthy fraction does — "no cheap attribution found", never "anonymous"). Auditing pools reaches auditors. Protecting a user at the moment they act reaches every user of every privacy protocol on the chain.

9 The coordinator: forming a good crowd on purpose

Here is the payoff of being able to measure. Every other privacy tool treats the crowd as given. RIVERRUN can *measure* a crowd's anonymity, so an agent can *form* a good one on purpose.

The rule is backwards from intuition until you have the formula, then obvious. A crowd funded from the *same* origin is **strong**: an adversary who learns "the actor's money traces to Coinbase" has learned nothing — everyone's does. A crowd of *distinct* origins is **weak**: learning the origin names the one member who has it. So the agent groups members who *share* an origin, and holds back the singletons who would be exposed. Diversity is the enemy here, not the friend.

The policy is a floor. `coordinate(pending, k_min)` admits a member only if at least $k_{\min}$ pending members share their provenance class, and defers the rest with a concrete remedy (wait for more same-origin members, or re-fund from an origin the round already has).

Property 9.1. Every admitted member ends in a class of size $\geq k_{\min}$, so no admitted member is exposed; and among all rounds with that property this one is the largest, since it admits every safe member.

Ten pending members: classes A=5, B=3, and two singletons C, D. Admit everyone (round of 10): $k_{\text{eff}} \approx 3.1$. Coordinate with $k_{\min}$=2 (round of 8, C and D deferred): $k_{\text{eff}} \approx 4.1$. **Fewer members, more anonymity, and nobody exposed.**

Run as an agent loop, intents arrive over time and the agent fires a round the moment a same-origin crowd is ready, holding the rest until they are safe — or forever, if their origin stays rare. Refusing to act, when acting would expose you, is the privacy-preserving choice, and an autonomous agent acting on-chain should make it.

10 Certificates you can check without trusting the agent

The coordinator's claim — "this round is private" — should not require trust. A member about to join has to know the agent computed honestly and was not buggy or adversarial. So each fired round carries a *proof-carrying certificate*.

The insight: a proof is not a moat, because a competent engineer can replicate it. What is a moat is making the proof **inescapable** (you cannot claim a bound the evidence does not support), **portable** (a small artifact a stranger checks alone), and **bound to the deployment** (tied to the exact thing it certifies). A certificate turns "trust me" into "check my work."

When the agent fires a round it emits a certificate carrying the admitted set's provenance-class histogram, the floor $k_{\min}$, and the round's k_{eff}. Anyone verifies it. **Inescapable:** verification recomputes k_{eff} from the histogram and checks $\min_c n_c \geq k_{\min}$, so a coordinator cannot certify a bound the histogram does not support — a tampered number fails (tested). **Portable:** the certificate is a small object a member, an auditor, or a settling program checks without re-tracing the graph. **Bound to the round:** a blake3 commitment ties it to the exact admitted set, so it cannot be replayed onto a different crowd (tested). The certificate carries only the histogram, never member identities: it proves the crowd meets the floor without revealing who is in which class.

11 On-chain: what runs, measured

A privacy tool nobody can afford to run is a paper, not a tool. So we measured.

set size	proof	prove	verify	setup artifacts
4	11,929 B	2.0 ms	0.20 ms	0 bytes
64	17,045 B	3.7 ms	0.35 ms	0 bytes
16,384	19,064 B	8.1 ms	0.43 ms	0 bytes

A set of 16,384 members proves in 8 ms. Proof size grows logarithmically: 4096× the members costs 1.6× the proof. The last column is the point: nothing to distribute, nothing to trust.

The real question is whether the *chain* can check the proof. If it can, no one is trusted; if not, someone is. There are two proof systems in play, and they land in two different places.

The Winterfell STARK (the research core). We built a Solana program that runs the real Winterfell verifier — FRI, Merkle openings, Fiat–Shamir, constraint evaluation — inside the runtime, and measured it. It runs, and it costs 3.77M compute units at k=4. That exceeds Solana's 1.4M per-transaction cap, so it needs execution chunked across transactions. Sweeping the FRI queries to the floor (4→1.57M CU) does not get under the cap even at a security level no one would ship. So single-transaction verification of *this* proof is not a tuning problem. It is a *field* problem: the 128-bit field is too big.

The M31 Circle STARK (the fix, now proven in a VM). The answer, argued in the last version of this paper as a roadmap, is now demonstrated. A Circle STARK over the 31-bit field M31 — the field production provers use [14], with an existing on-chain Solana verifier [15], post-quantum, no ceremony — shrinks both proof and cost dramatically.

Using the M31 Circle-STARK verifier of [15], we proved and verified RIVERRUN's relation — membership plus nullifier plus a bound public action — inside a Solana VM (LiteSVM). A real 8,696-byte proof is **accepted at** 159,849 **compute units**, about 11% of the 1.4M cap, in a single transaction. A proof carrying a *different* action is **rejected**. This is the whole verification, on-chain-class, under the cap.

Two honest limits. This ran in a local Solana VM (LiteSVM), not yet on mainnet, and the relation it verifies is riverrun's membership-plus-nullifier-plus-action, not yet the full in-circuit Poseidon2 Merkle path (the hash foundation is built on Plonky3's vetted parameters; the arithmetization is the next step). Until that lands on mainnet, the deployed program does not verify the proof itself. Rather than leave that implicit, it makes the trust explicit, bounded, and *distributed*: the pool names a committee of N verifier keys and a threshold M, and an execution must carry at least M signatures from *distinct* committee members over exactly the tuple being settled, checked through the runtime's signature precompile. This removes the two cheap attacks — an arbitrary signer settling an action, a watcher front-running a nullifier — and prices the rest: a false attestation now needs M colluding verifiers, not one. Five on-chain tests

> cover it (a 2-of-3 quorum settles, one vote alone does not, a member signing twice counts once, an outsider does not count). The M31 path above is how the trust goes to zero.

12 The threat model, plainly

A privacy claim without a threat model is not falsifiable, and an unfalsifiable claim is not an engineering claim. So here is the adversary RIVERRUN is built against.

What the adversary can do. See the entire public ledger forever — every transaction, amount, address, and the funding graph behind any wallet — run modern chain-analysis, and keep a tag database of exchange and service addresses. Observe the pool's public record $\{R, a, n\}$ for every execution. For the funding-graph results, also observe the provenance class of the *acting* identity, which is cheap because a fresh withdrawal wallet has to be funded from somewhere visible.

What RIVERRUN defends. The link between an actor and their action. Given an execution, the adversary should do no better than $1/k_{\text{eff}}$ at naming the author, where k_{eff} is the effective size of that member's provenance class — and the tools here *measure* k_{eff} rather than assume it.

What RIVERRUN does not defend, by design. Amounts and balances, which are public here (hiding them is a separate, composable problem). The value layer of consensus: the ledger records that value moved, and no circularity changes that; the circularity lives on the attribution layer an analyst reads, not the settlement layer consensus enforces. And, in the deployed program today, the correctness of the membership proof, delegated to the verifier committee until on-chain M31 verification lands on mainnet. Every number in this paper is stated against this adversary.

13 What does not hold yet

A paper that only lists its strengths is an advertisement. In one place, what RIVERRUN does not yet do:

- **The circuit is hand-rolled and unaudited.** Mutation testing removes worthless tests; it is not an audit.
- **It is not formally zero-knowledge.** The prover does not randomize the witness, so the honest claim is "the secret is not on the wire", checked empirically.
- **On-chain verification is proven in a local VM, not on mainnet.** The M31 relation verifies at 159,849 CU in LiteSVM; the deployed program still leans on the M-of-N committee until that ships to mainnet.
- **riverrun ID's in-circuit membership is not built.** The seven-power primitive is tested; the Poseidon2 Merkle proof over M31 and its Solana verifier are the road ahead.
- **Security parameters are example-grade** in the Winterfell path, roughly 84 bits, not the 128 a deployment needs.
- **"Rootless" provenance is conditional**: the circularity holds only while a construction's funding sources are themselves unattributable.
- **The ricorso proof is specified, not built**, for the set-size alignment reason of §7.
- **Anti-Sybil is priced, not prevented**: money still buys k, though the funding graph makes the

cheap version of the attack visible to the ruler.

The implementation is Rust, MIT-licensed, with 143 passing tests across the core, the STARK, the pool, the ruler, and the on-chain program, and clean static analysis. Every measurement is reproducible from the repository with one command.

14 Related work

RIVERRUN's *goal* — hiding which member of a set acted — is anonymous authentication, a mature field. On Ethereum the identity layer is **Semaphore**; riverrun ID is a post-quantum, Solana-native cousin. PrivDID [7] states the same session-unlinkability goal; anonymous-credential frameworks with predicate proofs [8, 9] live in the same threat model; PLUME [6] formalizes the nullifier. The *metric* is Serjantov and Danezis [1]. The *programme* of measuring advertised-versus-true anonymity is established on Ethereum [2, 3, 4, 5]. On-chain STARK verification on Solana has been done, both with Winterfell [11, 16] and with the M31 Circle-STARK verifier [15] we build on. What is narrow and new here is the combination: the payload is behavior rather than value; one secret becomes an unlinkable per-context identity on a chain with no Semaphore; and the funding graph is *measured* rather than assumed away, then turned into a user-facing pre-flight defense and a crowd-forming agent with checkable certificates.

15 Conclusion

Three things, stronger together than apart. A **ruler** that tells you what a privacy pool's anonymity is really worth — and that runs on Solana mainnet today, reporting its own reliability so a rate-limited network never reads as "private." An **identity layer** that turns one secret into a different, unlinkable identity in every app while keeping one-action-per-person, all from a single hash, so a quantum computer does not break it. And a **pool** that hides behavior instead of money, proven by a transparent post-quantum STARK whose on-chain verification we have now shown inside the Solana VM at 159,849 compute units.

The honest version of a privacy system ships its own strongest attack and reports what it finds. That is what we tried to build: a tool that measures the gap between the privacy a system advertises and the privacy it delivers — and turns that measurement into a defense.

The cypherpunks were right that no one grants you privacy; you build it or you go without. What that generation could not yet do was *prove* to you that what they built worked. RIVERRUN is one attempt to close that gap: privacy you do not have to take on faith, on a chain that will still be public and permanent long after the machine that threatens it exists. Build it post-quantum, ship your own strongest attack, and hand the reader the ruler. Then the claim is not "trust us." It is "check."

A Reproducing every number

The system is open-source and MIT-licensed. Everything runs from one script, `./demo.sh`. Individually:

claim	command
143 tests green	`cargo test -workspace`
riverrun ID's seven powers	`cargo test -p riverrun-core rotatable rln`
the turn, proven in zero knowledge	`cargo test -p riverrun-stark rotation`
one execution's cost	`cargo run -release -example bench`
the batched round	`cargo run -release -example behavior_pool`
the coordinator + certificate	`cargo run -release -example coordinator`
k_{eff} on RIVERRUN's designs	`riverrun exhibit`
k_{eff} of a live pool	`riverrun audit <POOL> 30`
your anonymity before you act	`riverrun preflight <WALLET>`
M31 verification inside the Solana VM	`cargo test -p ...stark-verifier -test cu`

The live-data commands read only public chain data, name no individual, and print their own bounds: a clean result is a floor — "no cheap attribution found" — never a proof of anonymity.

References

[1] A. Serjantov, G. Danezis. *Towards an Information Theoretic Metric for Anonymity.* PET 2002 (Outstanding Paper). https://bib.mixnetworks.org/pdf/serjantov2002towards.pdf

[2] N. Wu et al. *Tutela: Assessing User-Privacy on Ethereum and Tornado Cash.* arXiv:2201.06811. https://arxiv.org/abs/2201.06811

[3] F. Béres et al. *Blockchain is Watching You.* arXiv:2005.14051. https://arxiv.org/abs/2005.14051

[4] *Clustering Deposit and Withdrawal Activity in Tornado Cash.* arXiv:2510.09433 (2025). https://arxiv.org/abs/2510.09433

[5] H. Du et al. *Breaking the Anonymity of Ethereum Mixing Services Using Graph Feature Learning.* IEEE TIFS, 2024. https://doi.org/10.1109/TIFS.2023.3326984

[6] A. Gupta, K. Gurkan. *PLUME: An ECDSA Nullifier Scheme.* IACR ePrint 2022/1255. https://eprint.iacr.org/2022/1255

[7] *Toward Verifiable Privacy in Decentralized Identity (PrivDID).* IACR ePrint 2026/127. https://eprint.iacr.org/2026/127

[8] *Formalizing Privacy of Anonymous Credentials: A Framework with Predicate Proofs.* IACR ePrint 2026/1373. https://eprint.iacr.org/2026/1373

[9] *Re2creds: Reusable Anonymous Credentials.* IACR ePrint 2026/119. https://eprint.iacr.org/2026/119

[10] *Anonymous Self-Credentials and their Application to Single-Sign-On.* IACR ePrint 2025/618. https://eprint.iacr.org/2025/618

[11] J. Yano. *Full L1 On-Chain ZK-STARK+PQC Verification on Solana: A Measurement Study.* IACR ePrint 2025/1741. https://eprint.iacr.org/2025/1741

[12] A. Szepieniec, T. Ashur, S. Dhooghe. *Rescue-Prime: a Standard Specification (SoK).* IACR ePrint 2020/1143. https://eprint.iacr.org/2020/1143

[13] Facebook/Meta. *Winterfell: a STARK prover and verifier.* https://github.com/facebook/winterfell

[14] StarkWare. *Stwo: a Circle STARK prover over M31.* https://github.com/starkware-libs/stwo

[15] *Murkl: a Circle STARK verifier as a Solana CPI target.* https://github.com/exidz/murkl

[16] Wiener Labs. *Mosaic: on-chain verifier library for Solana (chunked FRI-STARK).* https://github.com/wienerlabs/mosaic